

TFTP Daemon Console

Windows 네이티브 데스크톱 애플리케이션 개발 및 UI 구현 작업 지시서

문서 버전: v1.0 대상 플랫폼: Windows 10 / 11 (x64) UI 디자인 기준: Claude Design System (다크/라이트 하이브리드 콘솔 레이아웃)

1. 프로젝트 개요 및 목적

본 작업 지시서는 네트워크 장비(Cisco, Extreme 등) 펌웨어 업그레이드 및 네트워크 부팅(PXE) 환경 구축을 위해 사용되는 **TFTP Daemon Console** 프로그램을 Windows 네이티브 환경(WPF 또는 C++/Win32/WinUI 3)으로 구축하기 위한 상세 가이드입니다.

제공된 Claude 디자인 레퍼런스를 바탕으로, 직관적인 대시보드 형태의 상태 모니터링, 실시간 로그 콘솔, 그리고 서버 설정 제어가 가능하도록 UI/UX 및 백엔드 스레드 구조를 정확히 구현해야 합니다.

2. UI/UX 레이아웃 구조 분석

Claude 디자인 시스템 기반의 메인 윈도우는 크게 **Top Toolbar (상태 및 제어)**, **Left Sidebar (설정 및 킷 스테이터스)**, **Main Content Area (실시간 트랜스퍼 및 로그 콘솔)**로 구성됩니다.

구역 (Section)	주요 컴포넌트	기능 및 동작 설명
Top Control Bar	Start/Stop 버튼, Status Badge, Port 표시	서버 구동 상태 제어 (Listening, Stopped, Error) 및 킷 액션 토글
Left Configuration Panel	Root Directory 선택, Port (기본 69), Timeout, Max Retries 입력창	TFTP 서버 실행 매개변수 설정 및 유효성 검증
Center Log & Console	RichTextBox / ListView (탭 구조: Live Logs / Active Transfers)	실시간 패킷 수신, 블록 전송 상태, 오류 로그를 타임스탬프와 함께 컬러링 출력
Bottom Status Bar	전송된 총 바이트 수, 활성 세션 수, 클라이언트 IP 목록	현재 연결된 클라이언트 정보 및 누적 통계 실시간 업데이트

3. 기술 스택 및 아키텍처 권장안

- **프레임워크:** WPF (.NET 8) 또는 WinUI 3 (현대적인 Fluent Design 적용 권장)
- **네트워크 코어:** 비동기 UDP 소켓 (``Socket.BeginReceiveFrom`` 또는 ``Socket.ReceiveAsync``) 기반 멀티스레드 TFTP 서버 엔진 (RFC 1350 준수 - RRQ/WRQ, DATA, ACK, ERROR 패킷 처리)
- **UI 바인딩:** MVVM 패턴 적용 (CommunityToolkit.Mvvm 활용)
- **로그 시스템:** 가상화된 리스트뷰 또는 고성능 로그 버퍼링 (실시간 대량 패킷 로깅 시 UI 멈춤 방지)

4. 상세 구현 지시 사항 (Tasks)

Task 1: 메인 윈도우 및 그리드 레이아웃 구축

- 윈도우 기본 크기: 1024x720 (최소 크기 800x600 고정 및 리사이징 지원)
- 테마: 최신 Windows 11 다크/라이트 모드 지원 (콘솔 영역은 가독성이 높은 다크 모던 테마 적용: 배경 #0f172a, 텍스트 #38bdf8 / #4ade80)
- 창 상단 타이틀바에 앱 아이콘 및 현재 바인딩된 IP/Port 실시간 표시

Task 2: 서버 제어 및 설정 패널 구현

- **Root Directory:** 폴더 찾아보기(FolderBrowserDialog) 버튼 및 경로 텍스트박스. 존재하지 않는 경로 선택 시 경고 메시지 출력.
- **Port 설정:** 기본값 69 (1024 이하 포트 사용 시 관리자 권한 실행 안내 툴팁 또는 경고 제공).
- **Start/Stop 버튼:** 서버 구동 성공 시 버튼 텍스트가 'Stop Server'로 변경되며 초록색 상태 표시등 점멸. 실패 시 상세 에러 메시지 팝업.

Task 3: TFTP 데몬 백엔드 엔진 연동

- **UDP 69번 포트 리스닝:** 클라이언트 요청(RRQ - 파일 다운로드, WRQ - 파일 업로드) 수신 시 독립된 Worker Thread 또는 Task 생성하여 블록(Block 512바이트 + 헤더 4바이트) 단위 데이터 스트리밍 처리.
- **타임아웃 및 재전송:** ACK 응답 지연 시 설정된 타임아웃(기본 3초) 내에 재전송 로직 구현.
- **보안 필터:** 루트 디렉토리 외부로 탈출하는 경로 조작(Path Traversal, 예: ../../) 시도 차단.

Task 4: 실시간 콘솔 로그 및 전송 세션 모니터링

- 로그 레벨별 색상 구분: **INFO** (연결 성공/파일 전송 완료), **TRANSFER** (블록 진행률), **WARN/ERROR** (타임아웃/파일 없음/권한 부족)
- 'Active Transfers' 탭에서 현재 다운로드/업로드 중인 클라이언트의 IP, 파일명, 전송률(KB/s), 진행률 바 (Progress Bar) 실시간 렌더링.

5. 빌드 및 패키징 (Deployment)

- **단일 실행 파일 패키징:** .NET Self-Contained 퍼블리시 기능을 사용하여 .NET 런타임이 설치되지 않은 환경에서도 단일 .exe 파일로 즉시 실행 가능하도록 빌드.

```
dotnet publish -c Release -r win-x64 --self-contained true -p:PublishSingleFile=true
```

- **방화벽 예외 처리:** 앱 최초 실행 시 Windows 방화벽 인바운드 허용(UDP Port 69) 알림 또는 자동 규칙 등록 가이드 기능 포함.